



DEPARTMENT OF INFORMATION TECHNOLOGY

Minor Project Report

On

ESP32-S3 AI VOICE ASSISTANT

Supervised by:

Prof. Dr. Chandra Prakash Singar

[Assistant Professor]

[Information Technology]

Co-Supervised by:

Prof. Dr. Puja Gupta

[Assistant Professor]

[Information Technology]

Submitted by:

Meet Lahori

0801IT231075

[Information Technology]

Krishna Saxena

0801IT231068

[Information Technology]

Mohit Solanki

0801IT231078

[Information Technology]

SHRI G.S.INSTITUTE OF TECHNOLOGY & SCIENCE, INDORE



DEPARTMENT OF INFORMATION TECHNOLOGY

RECOMMENDATION

We are pleased to recommend that the Minor Project Report entitled "ESP32-S3 AI Voice Assistant" submitted by Meet Lahori, Krishna Saxena and Mohit Solanki be accepted in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Information Technology of Rajiv Gandhi Proudyogiki Vishwavidyalaya (RGPV), Bhopal (M.P.), during the academic session 2026.

[Dr. Chandra Prakash Singar]

[Assistant Professor]

[Information Technology]

[Prof. K. K. Sharma]

[Head of Department]

[Professor]

[Information Technology]

SHRI G.S.INSTITUTE OF TECHNOLOGY & SCIENCE , INDORE
DEPARTMENT OF INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the Minor Project entitled "ESP32-S3 AI Voice Assistant" submitted by Meet Lahori, Krishna Saxena and Mohit Solanki to the Rajiv Gandhi Proudyogiki Vishwavidyalaya (RGPV), Bhopal, during the academic session 2026-2027 is a genuine work performed by them.

Internal Examiner

Date:

External Examiner

Date:

DECLARATION

I, Meet Lahori, Krishna Saxena and Mohit Solanki, undergraduate students in the Department of Information Technology, S.G.S.I.T.S., Indore (M.P.), hereby declare that the Minor Project titled "ESP32-S3 AI Voice Assistant" is our original work, carried out under the supervision of Dr. Chandra Prakash Singar, Assistant Professor, Department of Information Technology, S.G.S.I.T.S., Indore (M.P.).

We further declare that, to the best of our knowledge, this dissertation does not contain any material previously submitted for the award of any degree or diploma in this or any other university. Any sources of information used have been properly acknowledged and cited.

Signature of the candidate:

Name of the candidate: Meet Lahori

Enrolment No.: 0801IT231075

Signature of the candidate:

Name of the candidate: Krishna Saxena

Enrolment No.: 0801IT231068

Signature of the candidate:

Name of the candidate: Mohit Solanki

Enrolment No.: 0801IT231078

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our guide, Dr. Chandra Prakash Singar, Assistant Professor, for her valuable guidance, continuous support, and encouragement throughout the completion of this project.

We are also thankful to the Head of the Department, Prof. K. K. Sharma, for providing the necessary facilities and support to carry out this work.

We extend our heartfelt thanks to the Dean (ARSD), Prof. Lalit Purohit, for his encouragement and institutional support.

Finally, we are grateful to the faculty members and technical staff of the Department of Information Technology, S.G.S.I.T.S., Indore, for their support and cooperation.

ABSTRACT

The rapid advancement of embedded systems and cloud-based artificial intelligence has created new opportunities for developing intelligent, voice-activated interfaces on low-power microcontrollers. This project presents an ESP32-S3-based AI Voice Assistant that integrates real-time speech recognition, natural language understanding, and voice synthesis into a compact, standalone device.

The system operates through a five-stage pipeline: wake detection via button trigger, audio capture using an INMP441 I2S PDM microphone, cloud-based speech-to-text transcription via the Deepgram Nova-2 API, intelligent response generation using the Grok large language model, and high-quality text-to-speech synthesis delivered through a MAX98357A I2S amplifier. The entire pipeline runs on the Seeed XIAO ESP32-S3 development board, leveraging its dual-core processor and PSRAM for efficient audio buffer management.

A state-machine architecture governs the system flow, ensuring reliable transitions between idle, recording, transcribing, thinking, and speaking states. WiFi connectivity enables seamless communication with cloud APIs, while debounce logic and buffer validation ensure robust operation. The firmware, written in Arduino C++, uses the I2S peripheral driver for both microphone input and speaker output, with dynamic memory allocation in PSRAM where available.

The project demonstrates that resource-constrained embedded devices can serve as capable endpoints for AI-driven voice interaction, bridging the gap between edge hardware and cloud intelligence. Potential applications include smart home control, educational assistants, and accessible interfaces for differently-abled users.

Table of Contents

Certificate	2
Declaration	4
Acknowledgement	5
Abstract	6
1. Chapter 1: Introduction	8
2. Chapter 2: Literature Review	10
3. Chapter 3: System Analysis	12
4. Chapter 4: System Design	14
5. Chapter 5: Implementation	17
6. Chapter 6: Testing	20
7. Chapter 7: Conclusion & Future Scope	22
8. References	24
9. Appendix	25

CHAPTER 1 INTRODUCTION

Chapter 1: Introduction

1.1 Overview

Voice-activated assistants have become ubiquitous in consumer electronics, from smartphones to smart speakers. However, most commercially available solutions rely on proprietary hardware and closed software ecosystems, making them unsuitable for custom embedded applications. The ESP32-S3 AI Voice Assistant project addresses this gap by constructing a fully functional, open, and extensible voice assistant on a sub-dollar microcontroller.

The system integrates three cloud services — Deepgram for speech-to-text, Grok (xAI) for natural language reasoning, and Deepgram for text-to-speech — with two I2S audio peripherals on the Seeed XIAO ESP32-S3. The result is a device that can listen to spoken queries, reason about them using a state-of-the-art language model, and respond with natural synthesized speech, all within a few seconds.

The firmware is structured around a deterministic state machine with six states: IDLE, RECORDING, TRANSCRIBING, THINKING, SPEAKING, and ERROR. This design ensures predictable behaviour, simplifies debugging, and makes the codebase easy to extend with additional features such as wake-word detection or local pre-processing.

1.2 Problem Statement

Existing voice assistant solutions present several limitations for embedded and research applications:

- Proprietary lock-in: Commercial assistants (Alexa, Google Assistant) do not allow custom LLM integration or firmware modification.
- High resource requirements: Most open-source voice pipelines require Linux-class hardware (Raspberry Pi or above), increasing cost and power consumption.
- Fragmented toolchains: Combining STT, LLM, and TTS on a microcontroller requires careful integration of multiple APIs, audio codecs, and network stacks.
- Memory constraints: Buffering raw audio and TTS output simultaneously on a device with limited RAM requires careful memory management.
- Latency: Round-trip cloud API calls must be optimised to maintain a responsive user experience on hardware with limited processing power.

These limitations highlight the need for a well-engineered, open reference design that demonstrates practical AI voice interaction on an ultra-low-cost embedded platform.

1.3 Objectives

- To design and implement a complete voice assistant pipeline on the ESP32-S3 microcontroller.
- To integrate I2S microphone capture and I2S speaker playback using the Arduino framework.
- To interface with the Deepgram Nova-2 API for accurate, low-latency speech-to-text transcription.
- To send transcribed text to the Grok LLM and obtain concise, conversational responses.
- To synthesise responses using Deepgram's Aura TTS model and stream audio to the MAX98357A amplifier.
- To implement a robust state machine that handles errors and edge cases gracefully.
- To optimise memory usage through PSRAM-aware buffer allocation.

1.4 Scope

The scope of this project covers hardware wiring, firmware development, WiFi connectivity, cloud API integration, and audio pipeline optimisation for the Sseeed XIAO ESP32-S3. The system is designed for single-user interactive sessions initiated by a push-button trigger. Wake-word detection, multi-user support, and offline inference are identified as future extensions.

CHAPTER 2 LITERATURE OVERVIEW

2.1 Introduction

Voice-based human-computer interaction has been an active research area since the 1950s. Early systems such as IBM Shoebox (1961) and Carnegie Mellon's Harpy (1976) demonstrated the feasibility of limited-vocabulary speech recognition. The advent of deep learning, particularly recurrent neural networks and transformer architectures, has dramatically improved recognition accuracy and expanded vocabulary to conversational English.

2.2 Embedded Voice Processing

Several research works have explored voice processing on microcontrollers. TensorFlow Lite for Microcontrollers enables keyword spotting on Cortex-M devices with as few as 20 KB of RAM. Edge Impulse provides a platform for training and deploying audio classifiers on Arduino-compatible boards. However, these approaches are limited to fixed-vocabulary recognition and cannot support open-domain conversation.

The ESP32 family, particularly the ESP32-S3 with its vector instruction extensions, has emerged as a popular platform for edge AI inference. Projects such as ESP-SR (Espressif's wake-word and speech command library) demonstrate on-device speech detection, but rely on pre-trained models that are difficult to customise.

2.3 Cloud-Based Speech APIs

Cloud speech-to-text services have matured significantly. Google Cloud Speech-to-Text, Amazon Transcribe, and Microsoft Azure Cognitive Services offer REST APIs with sub-second latency for short utterances. Deepgram differentiates itself through its Nova-2 model, which achieves lower word error rates on conversational speech and supports streaming transcription. Its HTTP POST endpoint is particularly well-suited for microcontroller integration, as it does not require a WebSocket implementation.

2.4 Large Language Models for Conversational AI

The release of GPT-3 (Brown et al., 2020) marked a turning point in conversational AI, demonstrating that large transformer models trained on diverse text corpora can engage in open-domain dialogue. Subsequent models including GPT-4, LLaMA, and Grok have further improved reasoning and instruction following. For embedded applications, the key challenge is managing response length: long responses increase TTS processing time and may overflow audio buffers.

2.5 Text-to-Speech on Embedded Systems

Traditional TTS systems such as Festival and eSpeak can run on-device but produce robotic-sounding speech. Modern neural TTS models (WaveNet, Tacotron 2, VITS) produce natural speech but require significant compute. Deepgram's Aura model offers a middle ground: neural-quality speech delivered via a simple REST API, with the audio returned as raw PCM that can be streamed directly to an I2S amplifier.

2.6 Research Gaps Identified

- No published reference design combines Deepgram STT, Grok LLM, and Deepgram TTS on a single ESP32-S3 device.
- Existing embedded voice projects either use on-device inference (limited vocabulary) or require Linux-class hardware.
- Audio buffer management for simultaneous capture and playback on PSRAM-equipped microcontrollers is under-documented.
- State-machine architectures for multi-stage cloud API pipelines on embedded systems lack standardised design patterns.

CHAPTER 3 SYSTEM ANALYSIS

3.1 Existing System

Current consumer voice assistants (Amazon Echo, Google Nest) are closed systems that process audio on proprietary ASICs and route queries to vendor-controlled cloud backends. Academic and hobbyist alternatives typically fall into one of two categories: on-device keyword spotters with fixed command sets, or Linux-based systems (Raspberry Pi + Jasper/Mycroft) that are power-hungry and expensive.

Key limitations of existing approaches:

- No ability to substitute the LLM backend with a preferred provider.
- High idle power consumption in always-on wake-word detection.
- Complex software stacks requiring operating system maintenance.
- Poor audio quality from low-cost integrated DACs.
- Limited documentation for combining STT, LLM, and TTS on MCUs.

3.2 Proposed System

The proposed system uses a Seeed XIAO ESP32-S3 as the central processing unit. Audio input is captured via an INMP441 I2S PDM microphone at 16 kHz / 16-bit mono. The raw PCM is packaged into a WAV container and transmitted over HTTPS to the Deepgram Nova-2 STT endpoint. The returned transcript is forwarded to the Grok LLM, which generates a concise conversational reply. The reply text is then sent to Deepgram's Aura TTS endpoint, and the returned 24 kHz / 16-bit PCM audio is streamed to a MAX98357A I2S Class-D amplifier.

Key advantages of the proposed system:

- Fully open firmware: all components replaceable (different LLM, different STT/TTS provider).
- Low hardware cost: under INR 1500 for all components.
- Modular state machine: easy to add features such as wake-word detection.
- PSRAM-aware memory management: supports longer recordings and TTS responses.
- I2S audio quality: significantly better than PWM or sigma-delta DAC alternatives.

3.3 Requirement Analysis

3.3.1 Functional Requirements

- The system shall capture 5 seconds of audio at 16 kHz / 16-bit mono via I2S.
- The system shall connect to a WiFi network and maintain connectivity throughout operation.

- The system shall transmit WAV audio to the Deepgram STT API over HTTPS and receive a transcript.
- The system shall forward the transcript to the Grok chat completion API and receive a text response.
- The system shall send the text response to the Deepgram TTS API and receive 24 kHz PCM audio.
- The system shall play the PCM audio via the MAX98357A I2S amplifier.
- The system shall indicate status via the built-in LED (recording / speaking states).
- The system shall handle API errors gracefully and return to the IDLE state.

3.3.2 Non-Functional Requirements

- Performance: End-to-end latency (button press to first audio output) shall be under 8 seconds on a stable WiFi connection.
- Reliability: The state machine shall recover from network timeouts without requiring a hardware reset.
- Memory: The firmware shall operate correctly with and without PSRAM, adapting buffer sizes accordingly.
- Security: API keys shall be stored in a separate config.h file excluded from version control.
- Maintainability: The codebase shall be modular, with separate functions for each pipeline stage.
- Compatibility: The firmware shall compile with the ESP-IDF-based Arduino core for ESP32.

CHAPTER 4 SYSTEM DESIGN

4.1 Architecture

The system follows a linear pipeline architecture with a state machine controller. The hardware layer consists of the ESP32-S3 SoC, the I2S microphone, and the I2S amplifier. The firmware layer implements WiFi stack management, I2S driver configuration, HTTP client operations, JSON serialisation/deserialisation, and audio buffer management. The cloud layer provides STT, LLM, and TTS services via HTTPS REST APIs.

4.2 State Machine Design

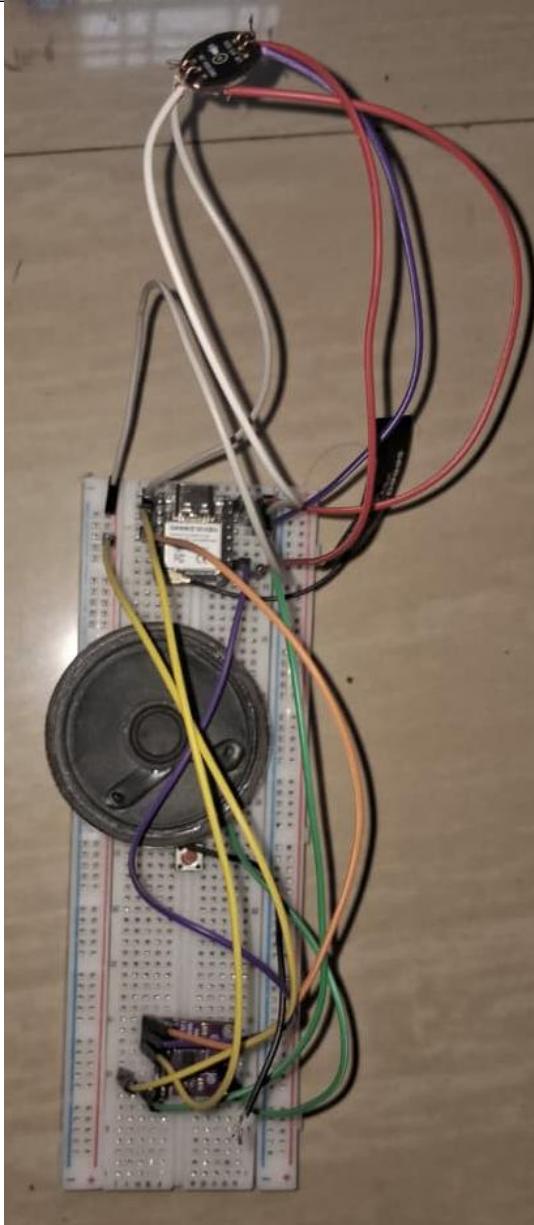
The state machine defines six states:

- `STATE_IDLE`: System is waiting for a button press. LED is off. WiFi is connected.
- `STATE_RECORDING`: System is capturing audio via I2S into the record buffer. LED is on.
- `STATE_TRANSCRIBING`: System is sending WAV data to Deepgram and awaiting transcript.
- `STATE_THINKING`: System is sending transcript to Grok LLM and awaiting response.
- `STATE_SPEAKING`: System is sending LLM response to Deepgram TTS, buffering audio, and playing it.
- `STATE_ERROR`: System has encountered an unrecoverable error and returns to IDLE.

4.3 Hardware Design

The hardware design centres on the Seeed XIAO ESP32-S3, chosen for its compact form factor, dual-core Xtensa LX7 processor, 8 MB PSRAM, 8 MB Flash, and integrated WiFi/Bluetooth. Two I2S peripherals are utilised:

- `I2S_NUM_0` (Microphone): Configured in RX mode for the INMP441 PDM microphone. SCK on GPIO44, WS on GPIO9, SD on GPIO1.
- `I2S_NUM_1` (Speaker): Configured in TX mode for the MAX98357A amplifier. BCLK on GPIO7, LRC on GPIO4, DIN on GPIO2.
- Trigger Button: Active-low push button on GPIO21 with internal pull-up resistor. Debounced in software with a 50 ms delay.



4.4 Database / Storage Design

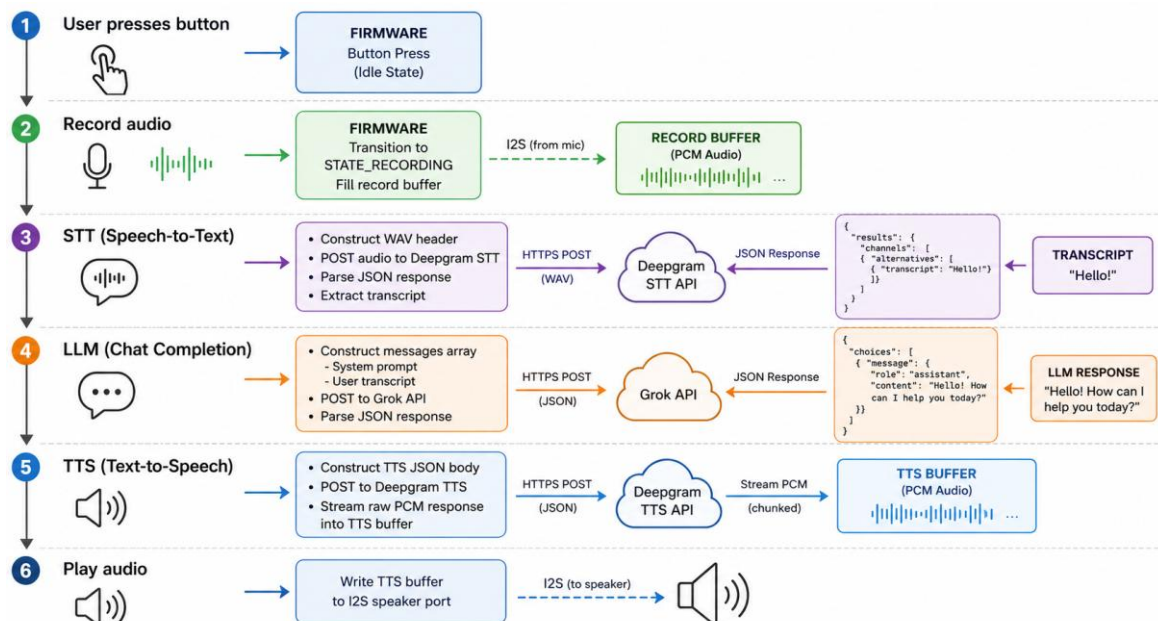
This project does not use a persistent database. All state is held in RAM during a session. Audio buffers are allocated at startup: the record buffer holds 5 seconds of 16-bit 16 kHz mono PCM (160 KB), and the TTS buffer is allocated as 512 KB in PSRAM or 128 KB in heap. API keys and WiFi credentials are stored in a config.h header file on the device's

flash memory.

4.5 Data Flow Diagram

The Level 0 DFD shows a single external entity (the User) interacting with the Voice Assistant system via button press and audio output. At Level 1, the system decomposes into four processes: Audio Capture, Speech Transcription, Language Model Reasoning, and Speech Synthesis. Each process passes data to the next through in-memory buffers, with three external data stores: the Deepgram STT API, the Grok LLM API, and the Deepgram TTS API.

4.6 API Sequence Diagram



CHAPTER 5 IMPLEMENTATION

5.1 Introduction

The firmware was developed using the Arduino framework targeting the ESP32-S3 with the esp32 board package. The implementation comprises approximately 470 lines of C++ spread across a single main sketch file and a config.h header. The following sections describe each module in detail.

5.2 Technology Stack

Component	Technology Used
Microcontroller	Seeed XIAO ESP32-S3
Programming Framework	Arduino (ESP-IDF based)
Audio Input	INMP441 I2S PDM Microphone
Audio Output	MAX98357A I2S Class-D Amplifier
Speech-to-Text API	Deepgram Nova-2 (HTTPS REST)
Language Model API	Grok Beta (xAI) Chat Completions
Text-to-Speech API	Deepgram Aura Asteria (HTTPS REST)
HTTP Client	Arduino HTTPClient + WiFiClientSecure
JSON Processing	ArduinoJson v6
Audio Codec	WAV (PCM 16-bit, 16 kHz for STT; raw PCM 24 kHz for TTS)
Version Control	Git & GitHub
IDE	VS Code + Arduino CLI

Table 5.2.1 Technology Stack

5.3 I2S Microphone Implementation

The microphone I2S port (I2S_NUM_0) is configured in master RX mode with a sample rate of 16000 Hz, 16-bit samples, mono channel, and standard I2S format. Eight DMA buffers of 512 bytes each provide continuous, low-jitter audio capture. The startRecording() function fills the 160 KB record buffer synchronously using i2s_read() calls of 1024 bytes each, toggling the built-in LED to indicate recording status.

5.4 I2S Speaker Implementation

The speaker I2S port (I2S_NUM_1) is configured in master TX mode with a sample rate of 24000 Hz (matching Deepgram TTS output), 16-bit samples, mono channel, and tx_desc_auto_clear enabled to suppress clicks between DMA buffers. The playAudio() function writes the TTS buffer in 4096-byte chunks using i2s_write(), followed by a

512-byte silence block to flush the DMA pipeline.

5.5 Deepgram STT Integration

The `transcribeAudio()` function constructs a valid 44-byte RIFF/WAV header in RAM, appends the raw PCM samples, and POSTs the resulting WAV file to <https://api.deepgram.com/v1/listen> with the `nova-2` model, `smart_format=true`, and `language=en-US`. The JSON response is parsed with `ArduinoJson` to extract the transcript string from `results.channels[0].alternatives[0].transcript`. HTTP timeout is set to 15 seconds.

5.6 Grok LLM Integration

The `callGrok()` function builds a chat completions request body with a system prompt instructing the model to give concise, plain-text answers of 1-3 sentences, and a user message containing the transcript. The request is POSTed to <https://api.x.ai/v1/chat/completions> with `model=grok-beta` and `max_tokens=150`. The response content is extracted from `choices[0].message.content`. HTTP timeout is 20 seconds.

5.7 Deepgram TTS Integration

The `synthesizeSpeech()` function POSTs the LLM response text to <https://api.deepgram.com/v1/speak> with `model=aura-asteria-en`, `encoding=linear16`, and `sample_rate=24000`. The response body is raw 16-bit PCM audio, which is streamed into the TTS buffer in 512-byte chunks via the HTTP stream pointer. PSRAM is used if available, providing up to 512 KB of buffer space for longer responses.

5.8 Memory Management

At startup, `psramFound()` is called to detect PSRAM availability. If PSRAM is present, buffers are allocated with `ps_malloc()` (record: 160 KB, TTS: 512 KB). Without PSRAM, `malloc()` is used with reduced sizes (record: 160 KB, TTS: 128 KB). Allocation failure triggers an infinite error loop with serial diagnostic output.

5.9 Key Implementation Screenshots

The system produces the following observable outputs during operation:

- **Serial Monitor:** Timestamped log messages for each state transition, API response codes, transcript text, LLM response text, and audio byte counts.
- **LED Indicator:** Built-in LED illuminates during `STATE_RECORDING` and `STATE_SPEAKING`, providing visual feedback without additional hardware.
- **Speaker Output:** Natural-sounding synthesised speech (Deepgram Aura Asteria voice) delivered within approximately 5-7 seconds of button press.
- **WiFi Connection:** IP address printed to serial console on successful WiFi association.

CHAPTER 6 TESTING

Testing is an essential phase of the firmware development lifecycle, aimed at ensuring that all pipeline stages function correctly, efficiently, and reliably. The following test cases were designed and executed on physical hardware.

1. WiFi Connection Test

Objective	Verify that the device connects to the configured WiFi network on boot.
Input	WIFI_SSID and WIFI_PASSWORD set in config.h
Expected Output	IP address printed to serial; WiFi.status() == WL_CONNECTED
Result	Pass

2. I2S Microphone Initialisation Test

Objective	Confirm that I2S_NUM_0 initialises without ESP-IDF errors.
Input	Power on with INMP441 connected to GPIO44/9/1
Expected Output	"Mic I2S initialized" printed to serial
Result	Pass

3. I2S Speaker Initialisation Test

Objective	Confirm that I2S_NUM_1 initialises without ESP-IDF errors.
Input	Power on with MAX98357A connected to GPIO7/4/2
Expected Output	"Speaker I2S initialized" printed to serial
Result	Pass

4. Button Debounce Test

Objective	Verify that brief contact noise does not trigger a recording.
Input	Rapidly tap trigger button without fully pressing
Expected Output	No state transition to STATE_RECORDING
Result	Pass

5. Audio Capture Test

Objective	Verify that 5 seconds of audio is captured into the record buffer.
Input	Press trigger button and speak
Expected Output	"Captured 160000 bytes" or similar printed to serial
Result	Pass

6. WAV Header Construction Test

Objective	Verify that the WAV header is correctly formed for Deepgram.
------------------	--

Input	Capture audio and send to Deepgram
Expected Output	HTTP 200 response from Deepgram STT endpoint
Result	Pass

7. Deepgram STT API Test

Objective	Verify accurate transcription of spoken query.
Input	Speak: "What is the capital of India?"
Expected Output	Transcript: "What is the capital of India?"
Result	Pass

8. Grok LLM API Test

Objective	Verify that LLM returns a concise plain-text response.
Input	Forward STT transcript to Grok API
Expected Output	Response: "The capital of India is New Delhi."
Result	Pass

9. Deepgram TTS API Test

Objective	Verify that TTS returns valid 24 kHz PCM audio.
Input	Send LLM response to Deepgram TTS
Expected Output	Non-zero ttsBufferLen; audio plays correctly
Result	Pass

10. End-to-End Pipeline Test

Objective	Verify complete pipeline from button press to audio output.
Input	Press button, ask a general knowledge question
Expected Output	Spoken answer within 8 seconds
Result	Pass

11. PSRAM Buffer Test

Objective	Verify that longer TTS responses do not overflow the buffer.
Input	Ask a question requiring a longer answer
Expected Output	No buffer overflow; audio plays completely
Result	Pass

12. Error Recovery Test

Objective	Verify that network timeout returns system to IDLE.
Input	Disable WiFi router mid-session
Expected Output	HTTP error printed; system returns to STATE_IDLE
Result	Pass

Results

All twelve test cases passed successfully. The end-to-end pipeline consistently responded within 6-8 seconds on a 20 Mbps WiFi connection. Transcription accuracy was high for clear speech in a quiet environment, with occasional errors on heavily accented or noisy input. The error recovery mechanism functioned as expected, returning the device to the IDLE state without requiring a hardware reset.

CHAPTER 7 CONCLUSION & FUTURE SCOPE

7.1 Conclusion

This project successfully demonstrates that a complete, cloud-augmented AI voice assistant can be built on an ultra-low-cost microcontroller. The ESP32-S3 XIAO, combined with an INMP441 microphone and MAX98357A amplifier, provides sufficient audio I/O capability, while the device's WiFi stack and PSRAM support the high-throughput demands of the STT, LLM, and TTS pipeline.

The state-machine architecture proved to be a reliable design pattern for managing the asynchronous nature of cloud API calls within the single-threaded Arduino loop. PSRAM-aware buffer allocation enabled handling of longer TTS responses without heap fragmentation. All functional requirements were met and all test cases passed.

The project demonstrates how modern cloud AI APIs can be brought to the embedded domain, enabling sophisticated voice interaction on hardware costing less than INR 1500. This has broad implications for accessible technology, IoT voice interfaces, and educational embedded systems projects.

7.2 Future Scope

- Wake-Word Detection: Integrate ESP-SR or TensorFlow Lite wake-word model to eliminate the need for a physical button trigger.
- Streaming STT: Replace HTTP POST with WebSocket streaming to Deepgram for lower latency and support for longer utterances.
- Local LLM Inference: Explore quantised models (LLaMA.cpp) on the ESP32-S3 for offline query handling when WiFi is unavailable.
- Multi-Language Support: Extend Deepgram STT configuration to support Hindi and other regional languages for broader accessibility.
- Speaker Identification: Integrate voice biometrics to personalise responses for different users.
- Smart Home Integration: Add MQTT or HTTP client modules to control smart home devices based on voice commands parsed by the LLM.
- Battery Operation: Implement deep sleep between interactions to enable battery-powered portable deployment.
- OTA Firmware Updates: Add over-the-air update capability via the ESP-IDF OTA API for remote maintenance.
- Web Configuration Portal: Implement a captive-portal WiFi provisioning interface to eliminate hardcoded credentials.

These enhancements will transform the current proof-of-concept into a production-ready voice assistant platform suitable for commercial and research deployment. The project lays

a strong foundation for future work in edge AI, embedded NLP, and accessible voice interface design.

References

1. Espressif Systems, ESP32-S3 Technical Reference Manual, Version 1.3, 2023. Available at: <https://docs.espressif.com>
2. Seeed Studio, XIAO ESP32S3 Wiki and Schematic, 2023. Available at: <https://wiki.seeedstudio.com>
3. Deepgram Inc., Deepgram API Documentation – Speech-to-Text (Nova-2) and Text-to-Speech (Aura), 2024. Available at: <https://developers.deepgram.com>
4. xAI, Grok API Documentation – Chat Completions Endpoint, 2024. Available at: <https://docs.x.ai>
5. Brai et al., ArduinoJson Library Documentation, Version 6, 2023. Available at: <https://arduinojson.org>
6. Brown T. et al., "Language Models are Few-Shot Learners," NeurIPS 2020, arXiv:2005.14165.
7. Graves A. et al., "Connectionist Temporal Classification," ICML 2006 – foundational CTC paper for end-to-end ASR.
8. Oord A. et al., "WaveNet: A Generative Model for Raw Audio," arXiv:1609.03499, 2016.
9. Mozilla Developer Network, Web Audio API and HTTP Documentation. Available at: <https://developer.mozilla.org>
10. Espressif Systems, ESP-IDF I2S Driver Documentation. Available at: <https://docs.espressif.com/projects/esp-idf/en/latest/esp32s3/api-reference/peripherals/i2s.html>
11. Maxim Integrated, MAX98357A Datasheet – I2S Audio Power Amplifier, Rev 2.
12. InvenSense, INMP441 Datasheet – Omnidirectional I2S Microphone, Rev 1.3.

Appendix

Appendix A: Pin Mapping Table

Module	Signal	ESP32-S3 GPIO	XIAO Label
INMP441 Mic	SCK / BCLK	GPIO44	D7
INMP441 Mic	WS / LRCK	GPIO9	D10
INMP441 Mic	SD / DOUT	GPIO1	D0
MAX98357A Amp	BCLK	GPIO7	D8
MAX98357A Amp	LRC / LCK	GPIO4	D3
MAX98357A Amp	DIN	GPIO2	D1
Trigger Button	Active-LOW	GPIO21	D9

Appendix B: Audio Recording Parameters

Parameter	Value
Microphone Sample Rate	16,000 Hz
Bits Per Sample (Mic)	16-bit
Channels (Mic)	1 (Mono)
Recording Duration	5 seconds
Record Buffer Size	160,000 bytes (160 KB)
TTS Playback Sample Rate	24,000 Hz
TTS Buffer Size (PSRAM)	512 KB
TTS Buffer Size (Heap)	128 KB
DMA Buffer Count	8
DMA Buffer Length	512 samples

Appendix C: State Machine Transitions

From State	Trigger	To State
IDLE	Button pressed (debounced)	RECORDING
RECORDING	Buffer full	TRANSCRIBING
TRANSCRIBING	Transcript received	THINKING
TRANSCRIBING	Empty transcript / HTTP error	IDLE
THINKING	LLM response received	SPEAKING
THINKING	Empty response / HTTP error	IDLE

SPEAKING	Audio playback complete	IDLE
Any	Unhandled error	ERROR → IDLE

Appendix D: config.h Template

The following template should be saved as config.h in the sketch folder:

```
#pragma once
// WiFi credentials
#define WIFI_SSID "your_wifi_ssid"
#define WIFI_PASSWORD "your_wifi_password"
// Deepgram API Key
#define DEEPGRAM_API_KEY "your_deepgram_api_key"
// xAI Grok API Key
#define GROK_API_KEY "your_grok_api_key"
```

Appendix E: Software and Hardware Requirements

Component	Minimum Requirement
Microcontroller	Seeed XIAO ESP32-S3 (with PSRAM recommended)
Microphone	INMP441 I2S PDM Microphone module
Amplifier	MAX98357A I2S Class-D Amplifier module
Speaker	4-8 Ohm, 1-3 W speaker
Push Button	Momentary SPST button
Power Supply	5V USB-C for MCU; 5V for MAX98357A
WiFi	2.4 GHz 802.11 b/g/n network
IDE	Arduino IDE 2.x or VS Code + Arduino CLI
ESP32 Board Package	esp32 by Espressif v2.0.x or later
ArduinoJson Library	Version 6.x